

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <math.h>

#define MAX 100

int stack[MAX];
int top = -1;

void push(int item) {
    if (top == MAX - 1) {
        printf("Stack overflow\n");
    } else {
        stack[++top] = item;
    }
}

int pop() {
    if (top == -1) {
        printf("Stack underflow\n");
        return 0;
    } else {
        return stack[top--];
    }
}

int evaluate_prefix(char *exp) {
    int len = strlen(exp);
    for (int i = len - 1; i >= 0; i--) {
        char ch = exp[i];

        if (isspace(ch))
            continue; // skip spaces

        if (isdigit(ch)) {
            push(ch - '0'); // convert char to int
        } else if (ch == '+' || ch == '-' || ch == '*' || ch == '/' || ch ==
'^') {
            int op1 = pop();
            int op2 = pop();

            switch (ch) {
                case '+':
                    push(op1 + op2);
                    break;
                case '-':
                    push(op1 - op2);
                    break;
                case '*':
                    push(op1 * op2);
                    break;
                case '/':
                    push(op1 / op2);
                    break;
                case '^':
                    push((int)pow(op1, op2));
                    break;
            }
        }
    }
    return pop();
}

```

```

int main() {
    char expr[MAX];

    printf("Enter a prefix expression: ");
    //fgets(expr, MAX, stdin);
    scanf("%s",&expr);
    expr[strcspn(expr, "\n")] = '\0'; // remove newline

    int result = evaluate_prefix(expr);
    printf("Result = %d\n", result);

    return 0;
}

```

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

```

```

#define MAX 100

```

```

char stack[MAX];
int top = -1;

```

```

// Function to push element to stack
void push(char c) {
    if (top == MAX - 1)
        printf("Stack Overflow\n");
    else
        stack[++top] = c;
}

```

```

// Function to pop element from stack
char pop() {
    if (top == -1)
        return -1;
    else
        return stack[top--];
}

```

```

// Function to get precedence of operators
int precedence(char c) {
    if (c == '^')
        return 3;
    else if (c == '*' || c == '/')
        return 2;
    else if (c == '+' || c == '-')
        return 1;
    else
        return 0;
}

```

```

// Function to convert infix to postfix
void infixToPostfix(char *infix, char *postfix) {
    int i, j = 0;
    char ch;
    for (i = 0; i < strlen(infix); i++) {
        ch = infix[i];

        if (isalnum(ch)) {
            // If operand, add to output
            postfix[j++] = ch;
        }
        else if (ch == '(') {

```

```

        push(ch);
    }
    else if (ch == ')') {
        // Pop until '('
        while (top != -1 && stack[top] != '(')
            postfix[j++] = pop();
        pop(); // remove '('
    }
    else {
        // Operator
        while (top != -1 && precedence(stack[top]) >= precedence(ch))
            postfix[j++] = pop();
        push(ch);
    }
}

// Pop remaining operators
while (top != -1)
    postfix[j++] = pop();

postfix[j] = '\0';
}

int main() {
    char infix[MAX], postfix[MAX];

    printf("Enter infix expression: ");
    scanf("%s", infix);

    infixToPostfix(infix, postfix);

    printf("Postfix expression: %s\n", postfix);

    return 0;
}

```